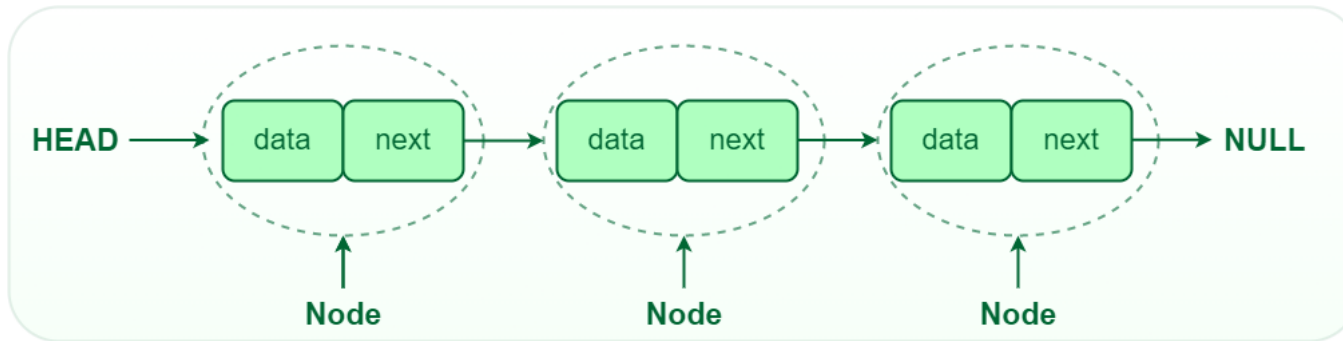


Lecture 10

Linked Lists



Announcements

- ❖ **Office hours** are up on course website
 - T: 12pm-4pm; F: 11am-1pm
 - Schedule an appointment individually or as a group
- ❖ **Project 1** Evaluation deliverable due **Friday @ 11:59pm**
 - See [example project](#) on Github
 - You received feedback via email
- ❖ **Midterm Exam** is next Tuesday (10/6) during lecture
 - 85% on paper (~40 minutes); 15% on computer (due at midnight)
 - Focus: empirical and theoretical comparisons of running time
 - Lab next Monday is a review exercise

Today

- ❖ Introduce new data structure: linked list
- ❖ Compare with array

Abstract Data Types

- ❖ Data type defined solely by its behavior from the user's point of view
- ❖ List (abstract data type)
 - Stores items in order
 - Get/insert/delete item at any index

Abstract Data Types

- ❖ Data type defined solely by its behavior from the user's point of view
- ❖ List (abstract data type)
 - Stores items in order
 - Get/insert/delete item at any index
- ❖ Python's implementation of a list (sometimes called an array)
 - Maintains an index where each index value points to an object
 - $O(1)$ operations: getting value at index; appending to the end
 - $O(n)$ operations: inserting at index; deleting at index

Insertion/Deletion in an Array

[42, 13, 17, 19, 23]

Insertion/Deletion in an Array

- ❖ Inserting into an array requires shifting elements down one index value
- ❖ $O(n)$ operation

[42, 13, 17, 19, 23]

[42, 11, 13, 17, 19, 23]



Insertion/Deletion in an Array

- ❖ Inserting into an array requires shifting elements down one index value

- ❖ $O(n)$ operation

[42, 13, 17, 19, 23]

- ❖ Deleting from an array requires shifting elements down one index value

- ❖ $O(n)$ operation

[42, 11, 13, 17, 19, 23]



[11, 13, 17, 19, 23]



New Task

- ❖ We want a list that can efficiently insert and delete

New Task

- ❖ We want a list that can efficiently insert and delete
- ❖ What property of an array is causing linear time insertion/deletion?

New Task

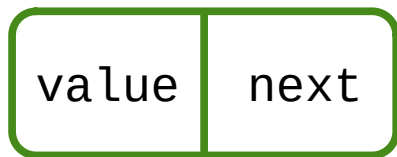
- ❖ We want a list that can efficiently insert and delete
- ❖ What property of an array is causing linear time insertion/deletion?
 - The index!
 - Can we build a list without an index?

Linked List

- ❖ A linked list is a list data structure
- ❖ Idea: utilizes pointers rather than an index

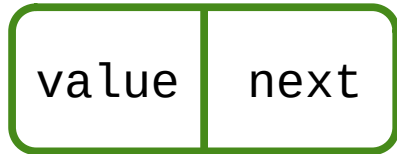
Linked List

- ❖ A linked list is a list data structure
- ❖ Idea: utilizes pointers rather than an index
- ❖ Data are stored in **nodes** that have two properties: **value** and **next**



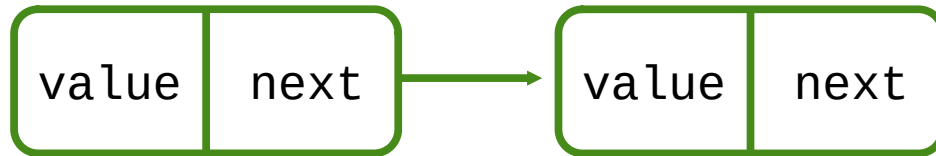
Linked List

- ❖ A linked list is a list data structure
- ❖ Idea: utilizes pointers rather than an index
- ❖ Data are stored in **nodes** that have two properties: **value** and **next**
 - value the item held by the list
 - next points to another node



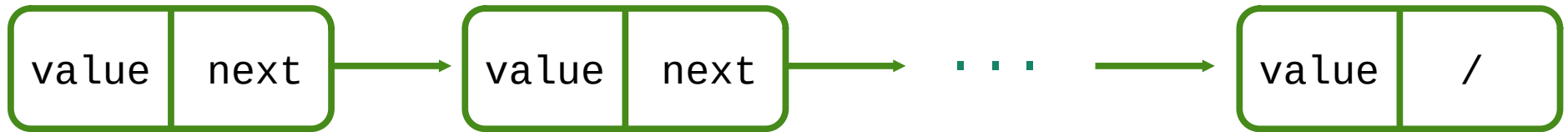
Linked List

- ❖ A linked list is a list data structure
- ❖ Idea: utilizes pointers rather than an index
- ❖ Data are stored in **nodes** that have two properties: **value** and **next**
 - value the item held by the list
 - next points to another node



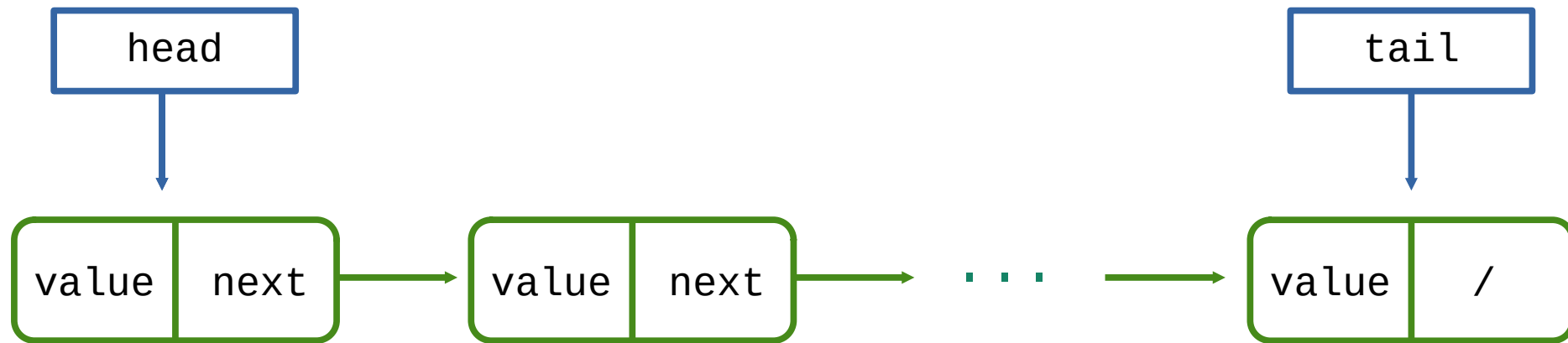
Linked List

- ❖ A chain of nodes



Linked List

- ❖ A chain of nodes
- ❖ Property **head** pointing to the first node in the chain
- ❖ Property **tail** pointing to the last node in the chain



Programming Perspective

- ❖ Implement node as a class with two properties: `value` and `next`

Programming Perspective

- ❖ Implement node as a class with two properties: `value` and `next`
 - `value` holds an integer, float, string, list, etc.
 - `next` holds another node or `None`

Programming Perspective

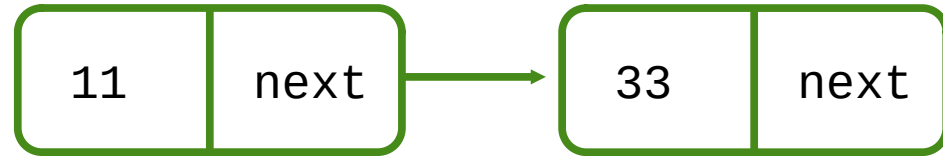
- ❖ Implement `node` as a class with two properties: `value` and `next`
 - `value` holds an integer, float, string, list, etc.
 - `next` holds another node or `None`
- ❖ Implement `LinkedList` as a class with two properties: `head` and `tail`

Programming Perspective

- ❖ Implement `node` as a class with two properties: `value` and `next`
 - `value` holds an integer, float, string, list, etc.
 - `next` holds another node or `None`
- ❖ Implement `LinkedList` as a class with two properties: `head` and `tail`
 - `head` and `tail` each hold a node

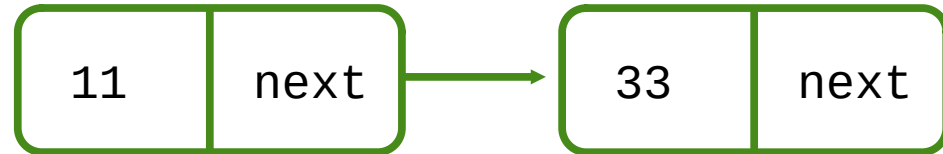
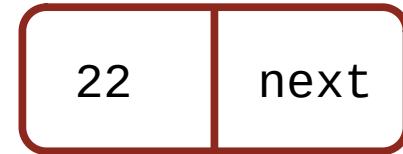
Insertion

- ❖ We have two nodes holding 11 and 33 respectively



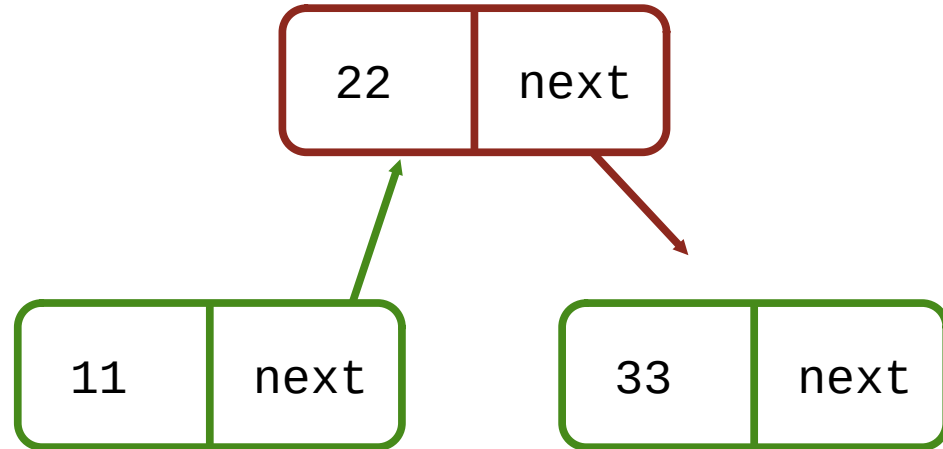
Insertion

- ❖ We have two nodes holding 11 and 33 respectively
- ❖ Want to insert a new node holding 22 between them



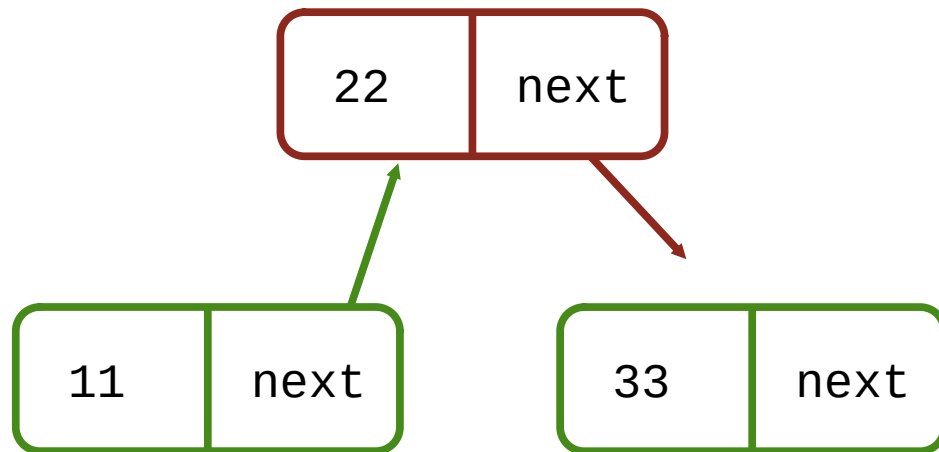
Insertion

- ❖ We have two nodes holding 11 and 33 respectively
- ❖ Want to insert a new node holding 22 between them
- ❖ Change the 11 node to point to the 22 node and the 22 node to point to the 33 node



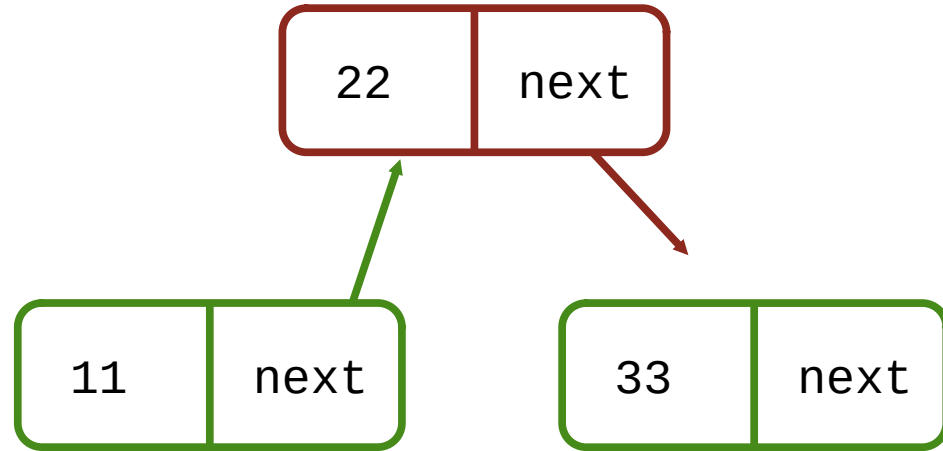
Insertion

- ❖ We have two nodes holding 11 and 33 respectively
- ❖ Want to insert a new node holding 22 between them
- ❖ Change the 11 node to point to the 22 node and the 22 node to point to the 33 node
- ❖ What is the running time of this operation?



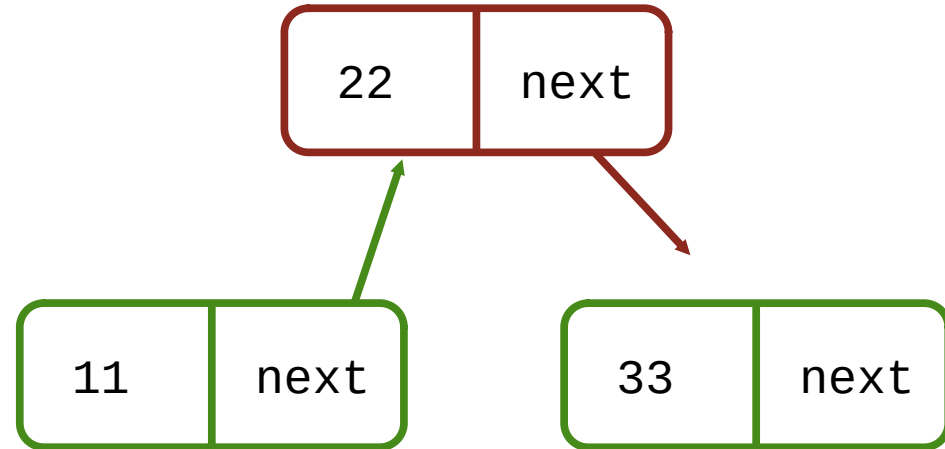
Insertion

- ❖ We have two nodes holding 11 and 33 respectively
- ❖ Want to insert a new node holding 22 between them
- ❖ Change the 11 node to point to the 22 node and the 22 node to point to the 33 node
- ❖ What is the running time of this operation?
 - $O(1)$



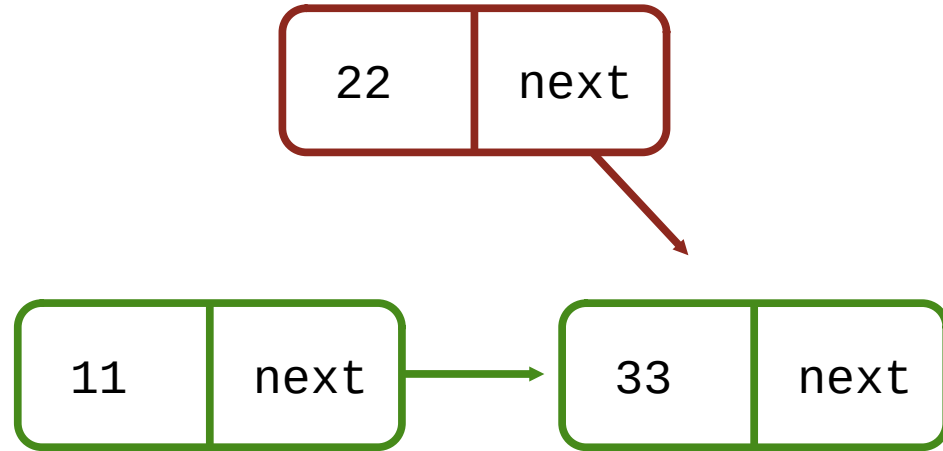
Deletion

- ❖ If we now wish to remove the 22 node, we just have to change what the 11 node points to



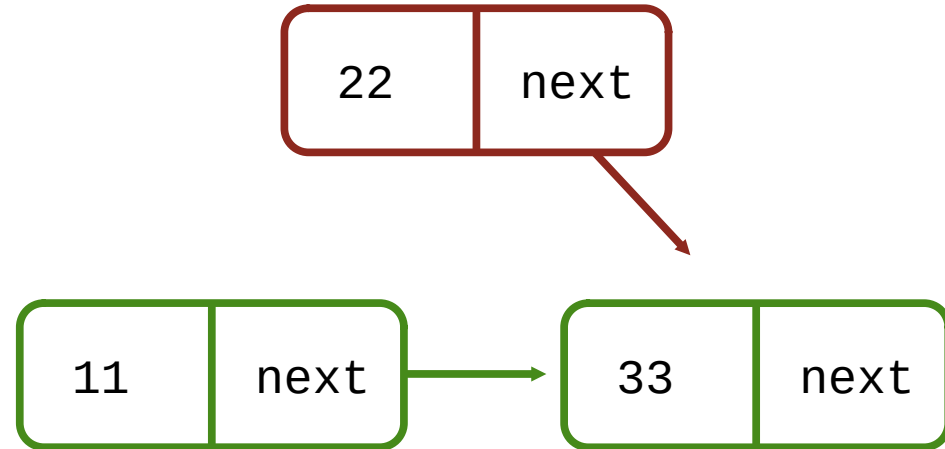
Deletion

- ❖ If we now wish to remove the 22 node, we just have to change what the 11 node points to
- ❖ This removes the 22 node from the chain!
- ❖ What is the running time of this operation?



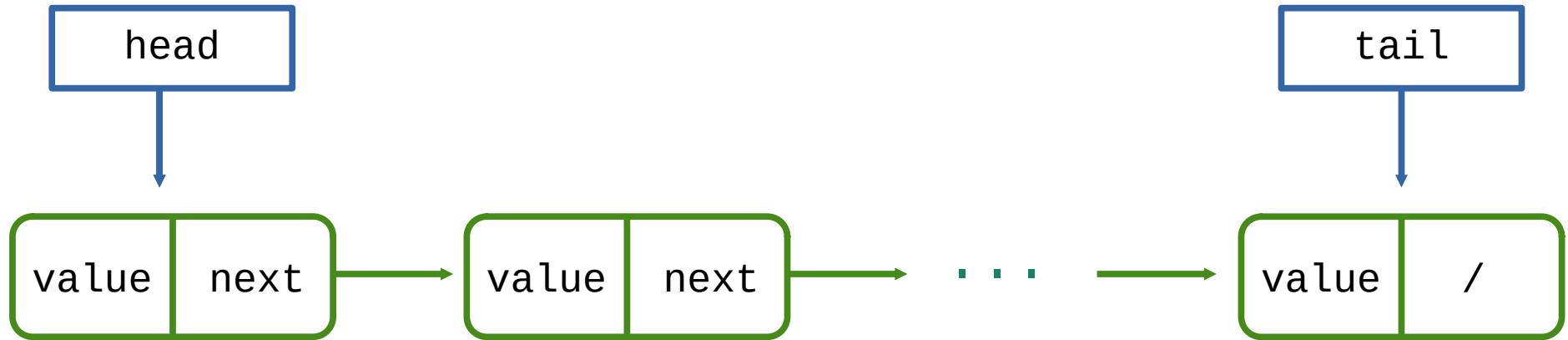
Deletion

- ❖ If we now wish to remove the 22 node, we just have to change what the 11 node points to
- ❖ This removes the 22 node from the chain!
- ❖ What is the running time of this operation?
 - $O(1)$



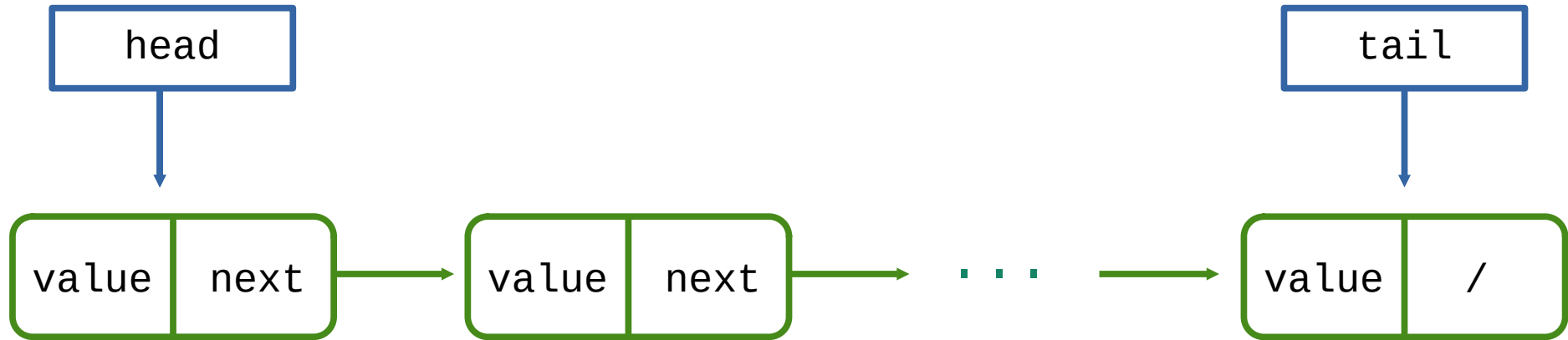
Getting

- ❖ Accessing a value in a linked list requires jumping nodes
- ❖ To get to the value at index 5 requires starting at the head and moving next five times



Getting

- ❖ Accessing a value in a linked list requires jumping nodes
- ❖ To get to the value at index 5 requires starting at the head and moving next five times
- ❖ Getting at an index is an $O(n)$ operation



Takeaways

- ❖ Linked List is a new list data structure
- ❖ Insertion and Deletion are $O(1)$
- ❖ Getting at an index is $O(n)$

Next Time

- ❖ Tradeoffs between arrays and linked lists
- ❖ Implementing a queue with a linked list
- ❖ DSP Ch. 7.2, 7.3